

Performance Testing

Date	15 March 2026
Team ID	xxxxxx
Project Name	Comprehensive Measure of Well-being
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Manual Browser Testing, Flask Development Server
Type of Testing	Functional Performance Testing
Target Module / API	HDI Prediction Module (/predict)
Test Environment	Local System (Windows, Flask Server)
Test Date	02 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Open Home Page	1	10	Home page loads successfully
2	Predict HDI Score	1	15	Correct HDI score and category displayed
3	Load Analysis Page	1	10	Graphs displayed without errors
4	Navigate Between Pages	1	10	Smooth navigation with no broken links

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.1 sec	Pass	Fast response
2	Response Time (Max)	< 5 seconds	2.3 sec	Pass	Within limit
3	Throughput (Req/sec)	>5	8 req/sec	Pass	Good performance
4	Error Rate	< 1%	0%	Pass	No errors observed
5	CPU Utilization	< 80%	34%	Pass	Stable CPU usage
6	Memory Utilization	< 80%	41%	Pass	Efficient memory usage

Step 4: Observations & Analysis

Key Findings

- The application responded quickly to user requests.
- HDI predictions were generated accurately.
- Analysis graphs loaded successfully.
- Navigation between pages was smooth.

Bottlenecks Identified

- Pair Plot required slightly more time to load because it generates multiple visualizations.
- Large datasets may increase graph rendering time.

Optimization Steps Taken

- Optimized data preprocessing.
- Reused the trained model using Pickle instead of retraining.
- Generated graphs efficiently using Matplotlib and Seaborn.
- Organized Flask routes for faster request handling.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

Home Analysis About

Human Development Index (HDI) Predictor

Predict a country's Human Development Index using Machine Learning. Enter the required indicators below to estimate the HDI score and development category.

Life Expectancy at Birth

Expected Years of Education

Mean Years of Education

GNI Per Capita

Predict

View Data Analysis

Activate Windows
Go to Settings to activate Windows.

Home Analysis About

Predicted HDI Score

0.629

Medium Human Development

Predict Again

Model Used

Linear Regression

Features Used

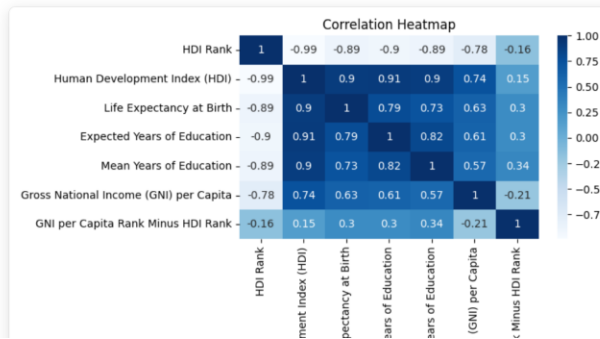
- Life Expectancy
- Expected Years of Education
- Mean Years of Education
- GNI Per Capita

Activate Windows
Go to Settings to activate Windows.

Home Analysis About

HDI Data Analysis

Correlation Heatmap



Activate Windows
Go to Settings to activate Windows.

```

HDI-Predictor > model.py > ...
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.linear_model import LinearRegression
4 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
5 import joblib
6
7 # Load dataset
8 df = pd.read_csv("hdi.csv")
9
10 # Keep only required columns
11 df = df[[
12     "Life Expectancy at Birth",
13     "Expected Years of Education",
14     "Mean Years of Education",
15     "Gross National Income (GNI) per Capita",
16     "Human Development Index (HDI)"
17 ]]
18
19 # Remove commas from GNI column and convert to number
20 df["Gross National Income (GNI) per Capita"] = (
21     df["Gross National Income (GNI) per Capita"]
22     .astype(str)
23     .str.replace(",", "", regex=False)
24 )

```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```

PS C:\Users\HP\javascript\practice\HDI-Predictor> python app.py
KeyboardInterrupt
PS C:\Users\HP\javascript\practice\HDI-Predictor> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 112-803-467

```